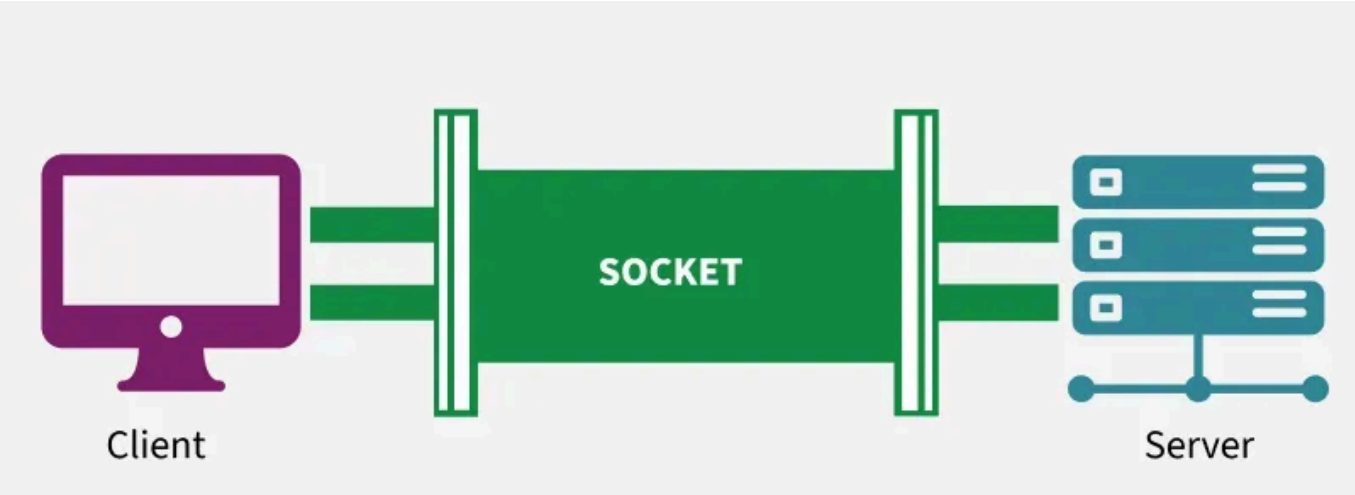
Lesson 4

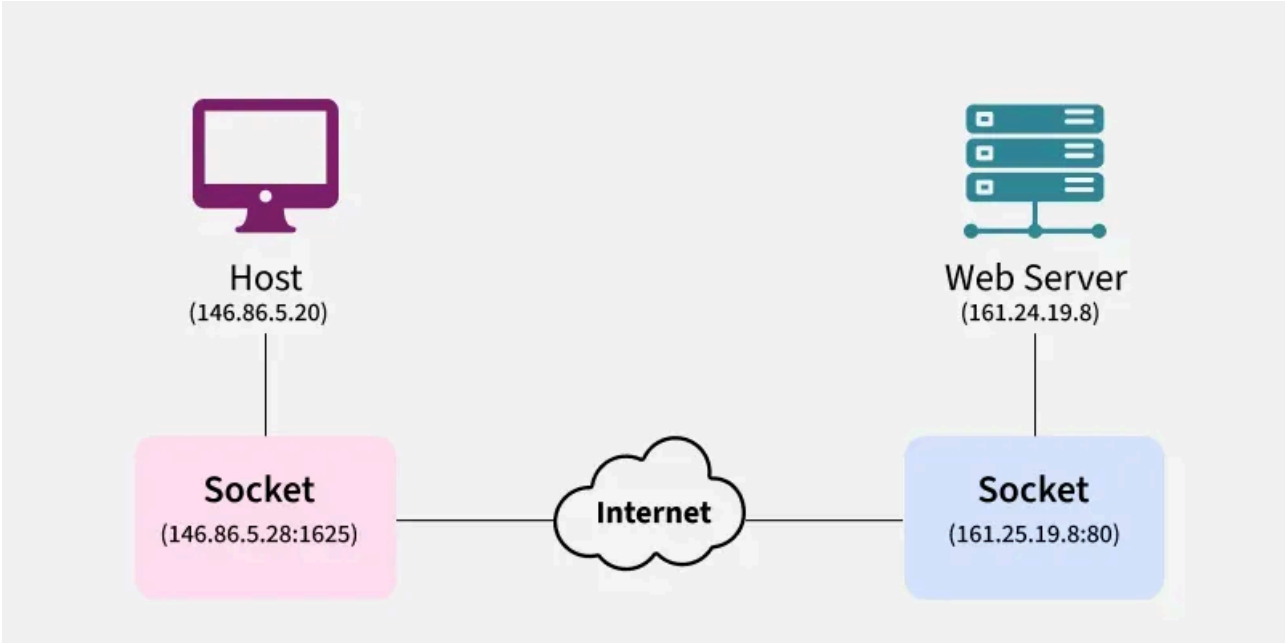
Sockets

What is a Socket?

- Sockets are a way to enable inter-process communication between programs running on a computer, or between programs running on separate computers. Programs that communicate via network sockets typically rely on using the Internet Protocol (IP) to send and receive data.
- It can also be identified as a software structure within a network node of a computer network that serves as an endpoint for sending and receiving data across the network.
- There are a number of different types of sockets. The most common are:
    1. **Stream sockets**:
       which use the Transmission Control Protocol (*TCP*) to encapsulate and ensure reliable delivery of a stream of data.
       These provide a *connection-oriented, sequenced*, and unique flow of data without record boundaries with *well defined mechanisms for creating and destroying connections and for detecting errors*.
    2. **Datagram sockets**:
       which use the User Datagram Protocol (*UDP*) to transmit datagrams, *without needing* to establish a persistent *connection* between systems.
    3. **Unix Domain Sockets**:
       which use local files to send and receive data instead of network interfaces and IP packets.
    4. **Raw sockets**:
       which allow applications to create and modify packets instead of relying on the host operating system.



- In the standard Internet protocols TCP and UDP, a socket address is the combination of an IP address and a port number.
- If we want to create multiple socket connections from the same host, we will be able to do so because OS will assign a different port number for each socket connection.
- That way we can have multiple connections between our host and the server.



- For the Host PC, the socket consists of the IP address 143.86.5.28, with the port number 1625; while the server socket has the IP address 161.25.19.8 with the port number 80(HTTP).

Key Port Number Ranges

- **Well-Known Ports (0–1023):** Reserved for core network services and system processes (e.g., FTP, SSH, HTTP, DNS).
- **Registered Ports (1024–49151):** Used by vendors for specific applications, such as Microsoft SQL Server (1433) or MySQL (3306).
- **Dynamic/Private Ports (49152–65535):** Used by client applications for outgoing connections, often called ephemeral ports

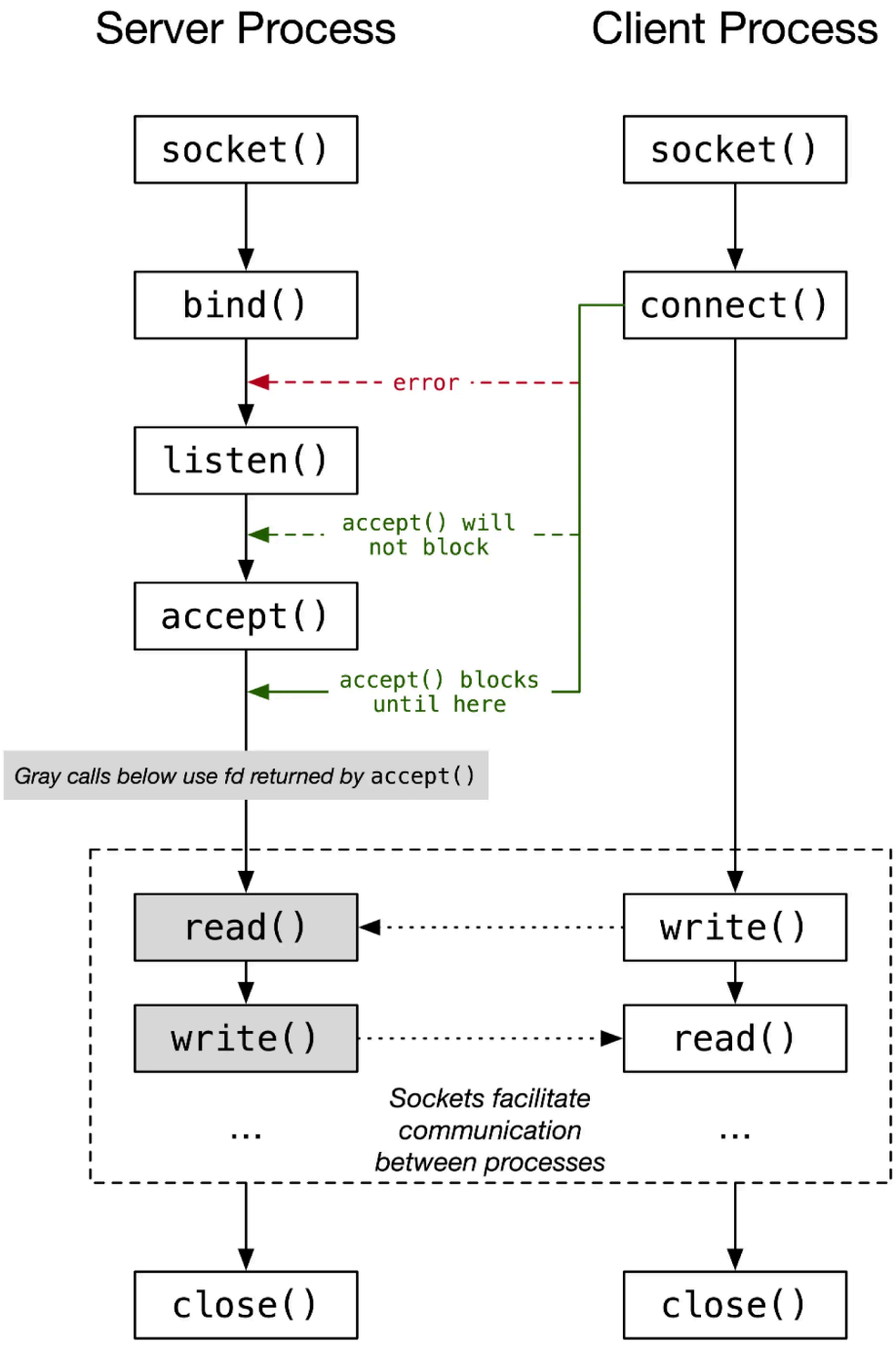
Common TCP and UDP Ports

- **20/21 (TCP):** File Transfer Protocol (FTP)
- **22 (TCP):** Secure Shell (SSH)
- **23 (TCP):** Telnet
- **25 (TCP):** Simple Mail Transfer Protocol (SMTP)
- **53 (TCP/UDP):** Domain Name System (DNS)
- **80 (TCP):** Hypertext Transfer Protocol (HTTP)
- **123 (UDP):** Network Time Protocol (NTP)
- **443 (TCP):** HTTP Secure (HTTPS

1. On a server, a process is *listening* on a port. Once it gets a connection, it hands it off to another thread. The communication never hogs the listening port.
2. Connections are uniquely identified by the OS by the following 5-tuple: (local-IP, local-port, remote-IP, remote-port, and protocol). If any element in the tuple is different, then this is a completely independent connection.
3. When a client connects to a server, it picks a *random, unused high-order source port*. This way, a single client can have up to ~64k connections to the server for the same destination port.

Socket Communication

State Diagram for Server and Client Model



SCALER Topics

- The nodes are divided into two types, **server** node and **client** node.
- The client node sends the connection signal and the server node receives the connection signal sent by the client node.

- Servers can accept session requests on listen() state.
- The **server** is **almost always in the listen() state**, and ready to receive connection requests.
- **The connection between a server and client node is established** using the socket over the **transport layer** of the internet.
- After a connection has been established, the client and server nodes can share information between them using the read and write commands.
- After sharing of information is done, the nodes terminate the connection.

Function Call	Description
Socket()	To create a socket
Bind()	It's a socket identification like a telephone number to contact
Listen()	Ready to receive a connection
Connect()	Ready to act as a sender
Accept()	Confirmation, it is like accepting to receive a call from a sender
Write()	To send data
Read()	To receive data
Close()	To close a connection

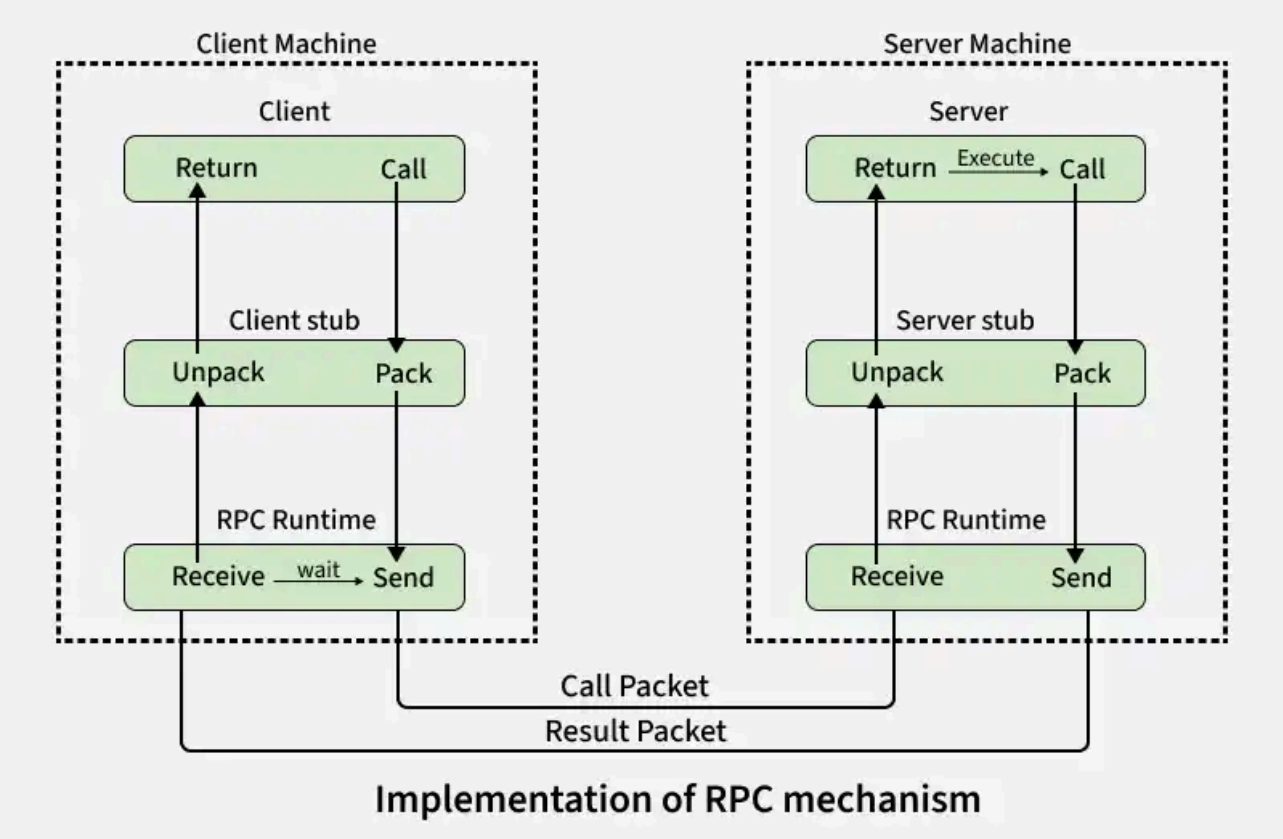
Advantages and Disadvantages of Sockets

Advantages

Disadvantages

Remote Procedure Call(RPC)

- Remote Procedure Call (RPC) is a way for a program to run a function on another computer in a network as if it were local.
- The client sends the request (with arguments) to the server, the server executes the function, and the result is sent back.
- RPC hides the details of networking, so developers can think in terms of normal function calls instead of complex network operations.



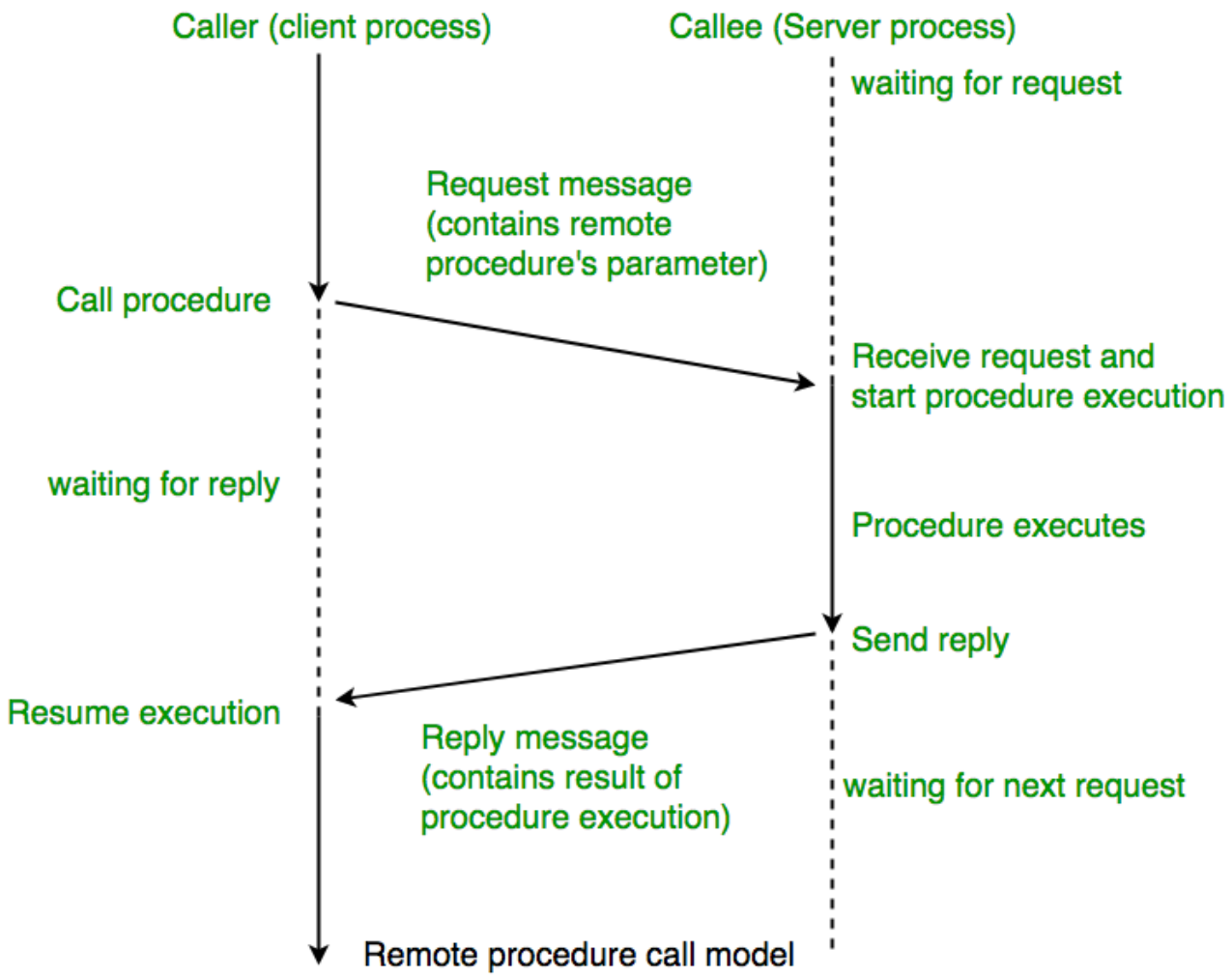
Implementation of RPC mechanism

How RPC Works (Step by Step)

1. **Client Calls Stub:** The client calls a local procedure (stub) as if it were normal.
2. **Marshalling**: The stub packs (marshals) all input parameters into a message.
3. **Send to Server**: The message is sent across the network to the server.(call packet)
4. **Server Stub:** The server stub unpacks the message and calls the actual server procedure.
5. **Execution & Return:** The server runs the procedure and returns the result to the stub.
6. **Back to Client:** The server stub sends the result back, and the client stub unpacks it.

Stub:

A helper code that hides the complexity from the programmer. On the client side, it converts function calls into messages (marshalling/unmarshalling) and works with the runtime to connect to the server. On the server side, the stub provides a similar interface between the run-time system and the local manager procedures that are executed by the server.



Advantages

- **Easy Communication:** RPC lets clients talk to servers using normal procedure calls in high-level programming languages. This makes it simple for programmers to work with.

- **Hidden Complexity:** RPC hides the details of how messages are sent between computers. This means programmers don't need to worry about the underlying network communication.
- **Flexibility:** RPC can be used in both local and distributed environments. This makes it versatile for different types of applications.

Disadvantages

- **Limited Parameter Passing:** RPC can only pass parameters by value. It can't pass pointers, which limits what can be sent between computers.
- **Slower Than Local Calls:** Remote procedure calls take longer than local procedure calls because they involve network communication.
- **Vulnerable to Failures:** RPC depends on network connections, other machines, and separate processes. This makes it more likely to fail than local procedure calls.